VELORA — Laravel 13 Enterprise Setup Guide (Part 1)

Steps 1–9: Install, Config, Auth, Folder Structure

STEP 1 — Install Laravel 13

```
# Pastikan PHP 8.3+ dan Composer terinstall
php -v
composer -V

# Install Laravel 13 via Composer
composer create-project laravel/laravel velora-api "^13.0"
cd velora-api

# Verifikasi versi
php artisan --version
```

Kenapa Laravel 13?

- Support PHP 8.3+ dengan fitur terbaru
- Built-in Reverb (WebSocket), Volt, Folio
- Improved queue, cache, dan pipeline performance

STEP 2 — Setup Environment

`.env` Production-Ready

```
APP_NAME=VELORA
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_TIMEZONE=Asia/Jakarta
APP_URL=http://localhost:8000
APP_LOCALE=id
APP_FALLBACK_LOCALE=en

# Database
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=velora_db
DB_USERNAME=root
DB_PASSWORD=secret
DB_CHARSET=utf8mb4
DB_COLLATION=utf8mb4_unicode_ci

# Cache & Queue
CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

# Redis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
REDIS_CLIENT=predis

# Mail
MAIL_MAILER=smtp
MAIL_HOST=smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS="noreply@velora.id"
```

```
MAIL_FROM_NAME="${APP_NAME}"

# Sanctum
SANCTUM_STATEFUL_DOMAINS=localhost,localhost:3000,127.0.0.1

# Logging
LOG_CHANNEL=daily
LOG_LEVEL=debug

# Velora Custom
VELORA_VERSION=1.0.0
VELORA_MAX_UPLOAD_SIZE=10240


php artisan key:generate
```

STEP 3 — Setup Sanctum

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

`config/sanctum.php` — Token Expiry

```
'expiration' => 60 * 24 * 30, // 30 hari
'token_prefix' => 'velora_',
'middleware' => [
    'authenticate_session' => Laravel\Sanctum\Http\Middleware\AuthenticateSession::class,
    'encrypt_cookies' => Illuminate\Cookie\Middleware\EncryptCookies::class,
    'validate_csrf_token' => Illuminate\Foundation\Http\Middleware\ValidateCsrfToken::class,
],
```

`bootstrap/app.php` — Daftarkan Sanctum Middleware

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->statefulApi();
})
```

STEP 4 — Setup Spatie Permission

```
composer require spatie/laravel-permission
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
php artisan migrate
```

`config/permission.php` — Key settings

```
'models' => [
    'permission' => Spatie\Permission\Models\Permission::class,
    'role'       => Spatie\Permission\Models\Role::class,
],
'table_names' => [
    'roles'                => 'roles',
    'permissions'          => 'permissions',
    'model_has_permissions' => 'model_has_permissions',
    'model_has_roles'      => 'model_has_roles',
    'role_has_permissions'  => 'role_has_permissions',
],
'cache' => [
    'expiration_time'  => \DateInterval::createFromDateString('24 hours'),
    'key'              => 'spatie.permission.cache',
    'store'            => 'redis',
],
```

User Model — Tambahkan Trait

```
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    use HasRoles;
    // ...
}
```

STEP 5 — Setup API Versioning

routes/api.php

```php
<?php

use Illuminate\Support\Facades\Route;

// Load semua module routes v1
Route::prefix('v1')->name('api.v1.')->group(function () {
    foreach (glob(app_path('Modules/*/Routes/api.php')) as $routeFile) {
        require $routeFile;
    }
});
```

bootstrap/app.php — Pastikan API route terdaftar

```php
->withRouting(
    api: __DIR__.'/../routes/api.php',
    apiPrefix: 'api',
    commands: __DIR__.'/../routes/console.php',
    health: '/up',
)
```

STEP 6 — Setup Response Helper

app/Shared/Helpers/ApiResponse.php

```php
<?php

namespace App\Shared\Helpers;

use Illuminate\Http\JsonResponse;
use Illuminate\Pagination\LengthAwarePaginator;

class ApiResponse
{
    public static function success(
        mixed $data = null,
        string $message = 'Success',
        int $statusCode = 200
    ): JsonResponse {
        return response()->json([
            'success' => true,
            'message' => $message,
            'data'    => $data,
            'meta'    => self::baseMeta(),
        ], $statusCode);
    }

    public static function created(mixed $data, string $message = 'Created successfully'): JsonR
    {
        return self::success($data, $message, 201);
    }

    public static function paginated(LengthAwarePaginator $paginator, mixed $data, string $messa
    {
        return response()->json([
            'success' => true,
            'message' => $message,
            'data'    => $data,
            'meta'    => array_merge(self::baseMeta(), [
                'current_page' => $paginator->currentPage(),
                'per_page'     => $paginator->perPage(),
                'total'        => $paginator->total(),
                'last_page'    => $paginator->lastPage(),
```

```
            'from'         => $paginator->firstItem(),
            'to'           => $paginator->lastItem(),
        ]),
    ], 200);
}

public static function error(
    string $message,
    int $statusCode = 400,
    string $errorCode = '',
    mixed $errors = null
): JsonResponse {
    $response = [
        'success' => false,
        'message' => $message,
        'meta'    => array_merge(self::baseMeta(), [
            'error_code' => $errorCode ?: 'ERROR_' . $statusCode,
        ]),
    ];

    if ($errors !== null) {
        $response['errors'] = $errors;
    }

    return response()->json($response, $statusCode);
}

public static function validationError(mixed $errors, string $message = 'Validation failed'
{
    return self::error($message, 422, 'VALIDATION_ERROR', $errors);
}

public static function notFound(string $message = 'Resource not found'): JsonResponse
{
    return self::error($message, 404, 'NOT_FOUND');
}

public static function unauthorized(string $message = 'Unauthorized'): JsonResponse
{
    return self::error($message, 401, 'UNAUTHORIZED');
}

public static function forbidden(string $message = 'Forbidden', string $errorCode = 'FORBIDI
```

```
    {
        return self::error($message, 403, $errorCode);
    }

    public static function noContent(): JsonResponse
    {
        return response()->json(null, 204);
    }

    private static function baseMeta(): array
    {
        return [
            'timestamp'  => now()->toIso8601String(),
            'request_id' => request()->header('X-Request-ID', uniqid('req_')),
        ];
    }
}
```

Register Helper — `composer.json`

```
"autoload": {
    "psr-4": {
        "App\\": "app/"
    },
    "files": [
        "app/Shared/Helpers/helpers.php"
    ]
}
```

app/Shared/Helpers/helpers.php

```php
<?php

if (!function_exists('api_success')) {
    function api_success(mixed $data = null, string $message = 'Success', int $code = 200) {
        return \App\Shared\Helpers\ApiResponse::success($data, $message, $code);
    }
}

if (!function_exists('api_error')) {
    function api_error(string $message, int $code = 400, string $errorCode = '') {
        return \App\Shared\Helpers\ApiResponse::error($message, $code, $errorCode);
    }
}

if (!function_exists('api_created')) {
    function api_created(mixed $data, string $message = 'Created successfully') {
        return \App\Shared\Helpers\ApiResponse::created($data, $message);
    }
}

if (!function_exists('api_paginated')) {
    function api_paginated(\Illuminate\Pagination\LengthAwarePaginator $paginator, mixed $data,
        return \App\Shared\Helpers\ApiResponse::paginated($paginator, $data, $message);
    }
}
```

```
composer dump-autoload
```

STEP 7 — Setup Exception Handler

app/Exceptions/Handler.php

```php
<?php

namespace App\Exceptions;

use App\Shared\Helpers\ApiResponse;
use Illuminate\Auth\AuthenticationException;
use Illuminate\Database\Eloquent\ModelNotFoundException;
use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
use Illuminate\Http\JsonResponse;
use Illuminate\Validation\ValidationException;
use Spatie\Permission\Exceptions\UnauthorizedException;
use Symfony\Component\HttpKernel\Exception\AccessDeniedHttpException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
use Throwable;

class Handler extends ExceptionHandler
{
    protected $dontFlash = [
        'current_password',
        'password',
        'password_confirmation',
    ];

    public function register(): void
    {
        $this->reportable(function (Throwable $e) {
            //
        });
    }

    public function render($request, Throwable $e): JsonResponse|\Illuminate\Http\Response
    {
        if ($request->expectsJson() || $request->is('api/*')) {
            return $this->handleApiException($e);
        }

        return parent::render($request, $e);
    }
```

```php
    private function handleApiException(Throwable $e): JsonResponse
    {
        return match (true) {
            $e instanceof ValidationException
                => ApiResponse::validationError($e->errors(), $e->getMessage()),

            $e instanceof ModelNotFoundException, $e instanceof NotFoundHttpException
                => ApiResponse::notFound('Resource not found'),

            $e instanceof AuthenticationException
                => ApiResponse::unauthorized('Unauthenticated. Please login.'),

            $e instanceof UnauthorizedException, $e instanceof AccessDeniedHttpException
                => ApiResponse::forbidden('You do not have permission to perform this action.'),

            $e instanceof \App\Shared\Exceptions\VeloraException
                => ApiResponse::error($e->getMessage(), $e->getStatusCode(), $e->getErrorCode()),

            default => ApiResponse::error(
                app()->isProduction() ? 'Internal server error.' : $e->getMessage(),
                500,
                'INTERNAL_ERROR'
            ),
        };
    }
}
```

app/Shared/Exceptions/VeloraException.php

```php
<?php

namespace App\Shared\Exceptions;

use RuntimeException;

class VeloraException extends RuntimeException
{
    public function __construct(
        string $message,
        private readonly int $statusCode = 400,
        private readonly string $errorCode = 'VELORA_ERROR',
    ) {
        parent::__construct($message);
    }

    public function getStatusCode(): int { return $this->statusCode; }
    public function getErrorCode(): string { return $this->errorCode; }

    public static function notFound(string $resource = 'Resource'): static
    {
        return new static("{$resource} not found.", 404, 'NOT_FOUND');
    }

    public static function forbidden(string $errorCode = 'FORBIDDEN'): static
    {
        return new static('Access denied.', 403, $errorCode);
    }
}
```

STEP 8 — Clean Architecture Folder Structure

```
# Buat folder struktur
mkdir -p app/Modules
mkdir -p app/Shared/Helpers
mkdir -p app/Shared/Traits
mkdir -p app/Shared/Middleware
mkdir -p app/Shared/Enums
mkdir -p app/Shared/Exceptions
mkdir -p app/Shared/Resources
mkdir -p app/Shared/Contracts
mkdir -p app/Providers
```

Powershell (Windows):

```
$dirs = @(
  "app/Modules",
  "app/Shared/Helpers",
  "app/Shared/Traits",
  "app/Shared/Middleware",
  "app/Shared/Enums",
  "app/Shared/Exceptions",
  "app/Shared/Resources",
  "app/Shared/Contracts"
)
foreach ($d in $dirs) { New-Item -ItemType Directory -Force -Path $d }
```

Struktur tiap Module

```
app/Modules/{ModuleName}/
├── Controllers/
├── Models/
├── Services/
├── Repositories/
│   ├── Contracts/
│   └── Eloquent/
├── DTOs/
├── Requests/
├── Resources/
├── Events/
├── Listeners/
├── Policies/
├── Enums/
├── Exceptions/
└── Routes/
    └── api.php
```

STEP 9 — Setup Repository Pattern

Interface Base —

app/Shared/Contracts/BaseRepositoryInterface.php

```php
<?php

namespace App\Shared\Contracts;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Pagination\LengthAwarePaginator;

interface BaseRepositoryInterface
{
    public function findById(int|string $id): ?Model;
    public function findOrFail(int|string $id): Model;
    public function all(array $filters = []): LengthAwarePaginator;
    public function create(array $data): Model;
    public function update(int|string $id, array $data): Model;
    public function delete(int|string $id): bool;
}
```

Abstract Base Repository —

app/Shared/Repositories/BaseRepository.php

```php
<?php

namespace App\Shared\Repositories;

use App\Shared\Contracts\BaseRepositoryInterface;
use App\Shared\Exceptions\VeloraException;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Pagination\LengthAwarePaginator;

abstract class BaseRepository implements BaseRepositoryInterface
{
    public function __construct(protected Model $model) {}

    public function findById(int|string $id): ?Model
    {
        return $this->model->find($id);
    }

    public function findOrFail(int|string $id): Model
    {
        return $this->model->findOrFail($id);
    }

    public function all(array $filters = []): LengthAwarePaginator
    {
        return $this->model->newQuery()
            ->paginate($filters['per_page'] ?? 15);
    }

    public function create(array $data): Model
    {
        return $this->model->create($data);
    }

    public function update(int|string $id, array $data): Model
    {
        $record = $this->findOrFail($id);
        $record->update($data);
        return $record->fresh();
    }
```

```
    public function delete(int|string $id): bool
    {
        $record = $this->findOrFail($id);
        return $record->delete();
    }
}
```

app/Providers/RepositoryServiceProvider.php

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;

class RepositoryServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // Binding akan ditambahkan per-module
        // Contoh:
        // $this->app->bind(
        //     \App\Modules\Tenant\Repositories\Contracts\BusinessRepositoryInterface::class,
        //     \App\Modules\Tenant\Repositories\Eloquent\BusinessRepository::class,
        // );
    }
}
```

Daftarkan di `bootstrap/app.php` :

```
->withProviders([
    App\Providers\RepositoryServiceProvider::class,
])
```